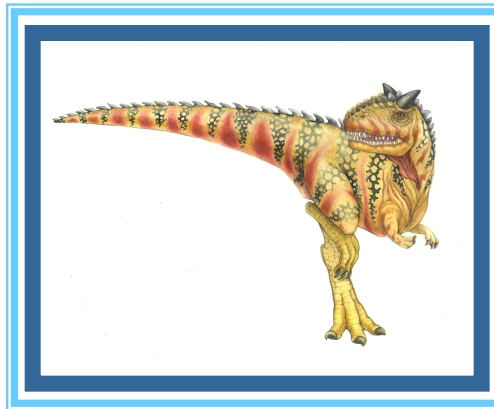


Chapter 8: Main Memory





Chapter 8: Memory Management

- Background
- Swapping
- Contiguous Memory Allocation
- Segmentation
- Paging
- Structure of the Page Table
- Example: The Intel 32 and 64-bit Architectures
- Example: ARM Architecture





Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques, including paging and segmentation
- To provide a detailed description of the Intel Pentium, which supports both pure segmentation and segmentation with paging





Background

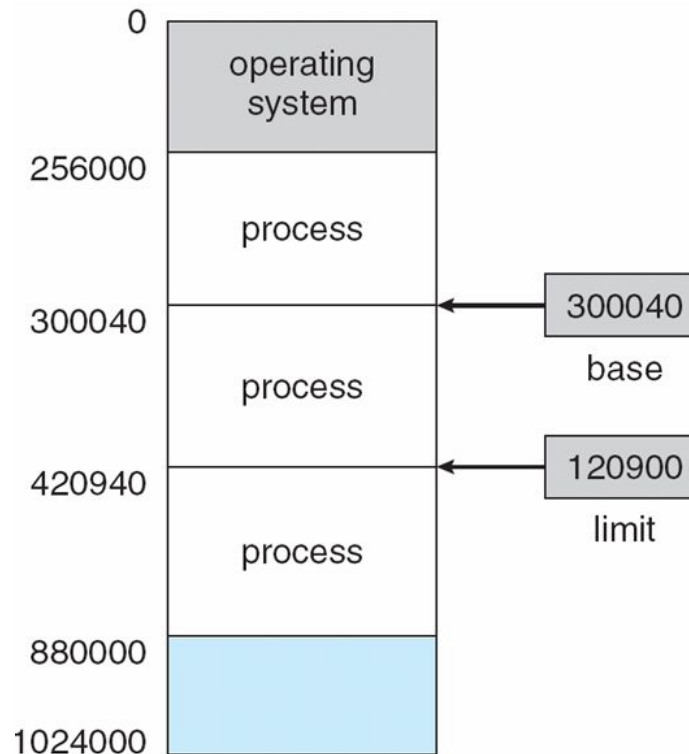
- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory is required to ensure correct operation





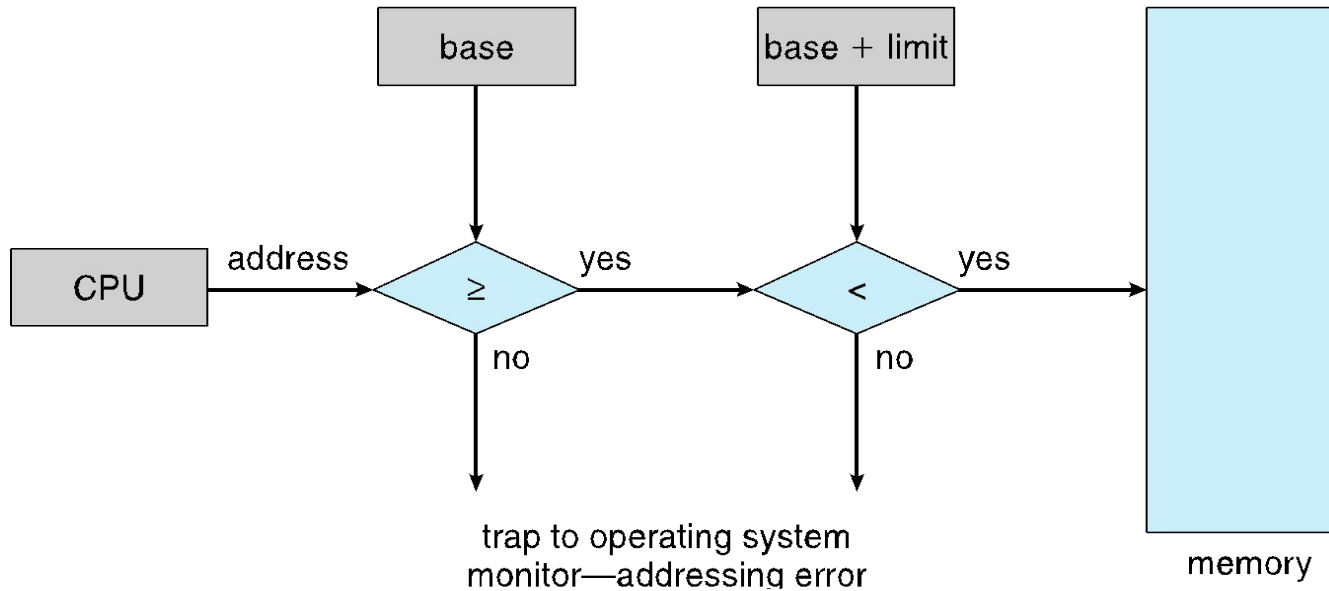
Base and Limit Registers

- A pair of **base** and **limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



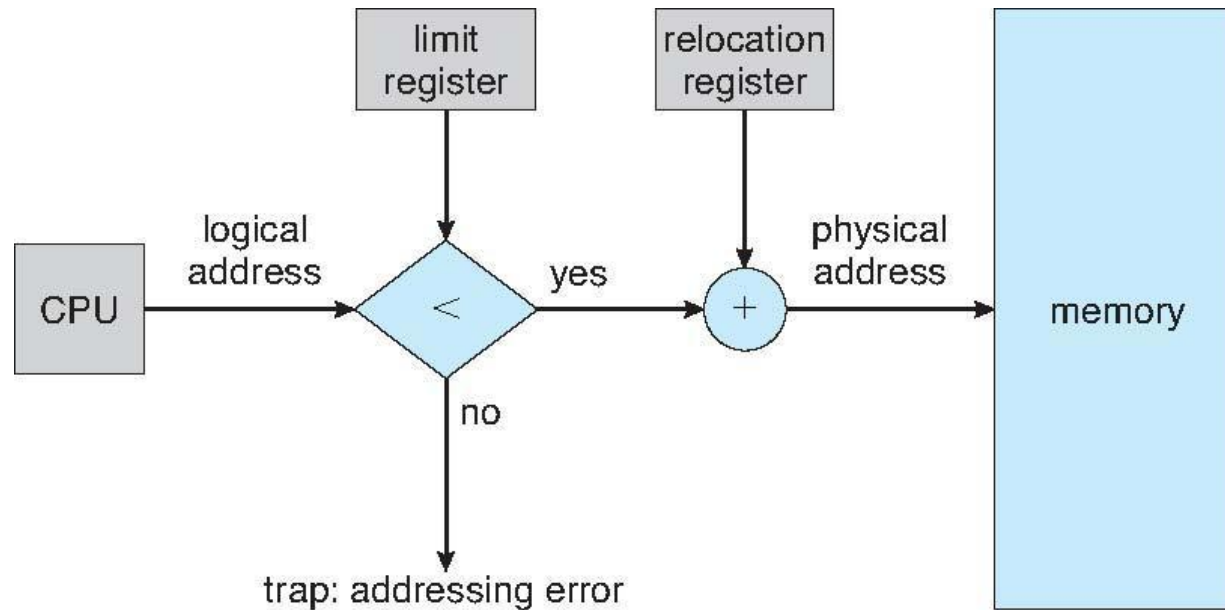


Hardware Address Protection





Hardware Support for Relocation and Limit Registers





Address Binding

- Programs on disk, ready to be brought into memory to execute form an **input queue**
 - Without support, must be loaded into address 0000
- Inconvenient to have first user process physical address always at 0000
 - How can it not be?
- Further, addresses represented in different ways at different stages of a program's life
 - Source code addresses usually symbolic
 - Compiled code addresses **bind** to relocatable addresses
 - 4 i.e. "14 bytes from beginning of this module"
 - Linker or loader will bind relocatable addresses to absolute addresses
 - 4 i.e. 74014
 - Each binding maps one address space to another





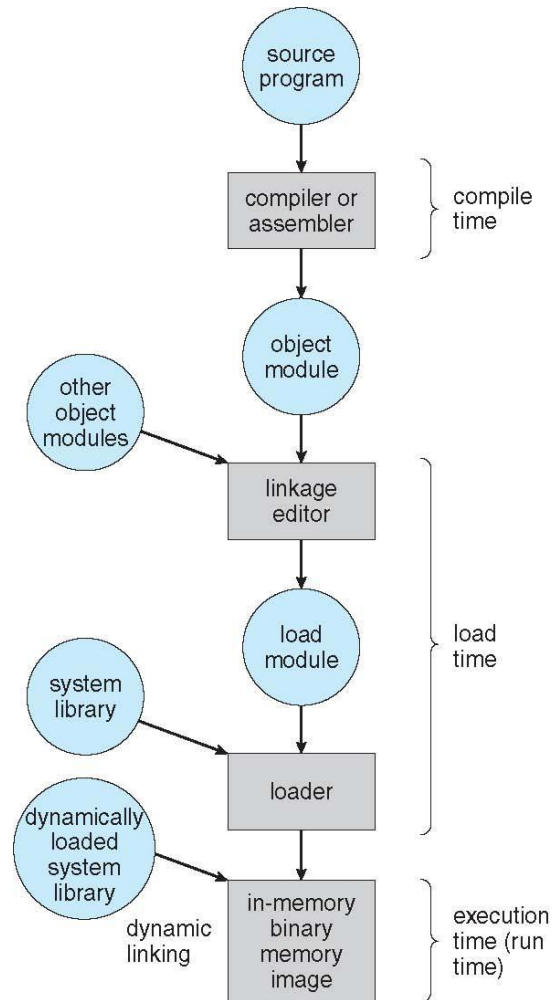
Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another
 - 4 Need hardware support for address maps (e.g., base and limit registers)





Multistep Processing of a User Program





Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address** – generated by the CPU; also referred to as **virtual address**
 - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program





Memory-Management Unit (MMU)

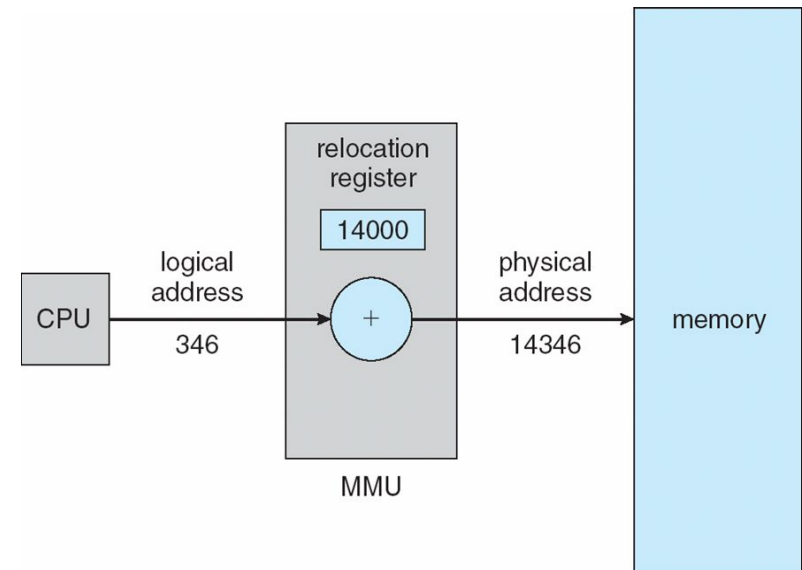
- Hardware device that at run time maps virtual to physical address
- Many methods possible, covered in the rest of this chapter
- To start, consider simple scheme where the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
 - Base register now called **relocation register**
 - MS-DOS on Intel 80x86 used 4 relocation registers
- The user program deals with *logical* addresses; it never sees the *real* physical addresses
 - Execution-time binding occurs when reference is made to location in memory
 - Logical address bound to physical addresses





Dynamic relocation using a relocation register

- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
 - Implemented through program design
 - OS can help by providing libraries to implement dynamic loading





Dynamic Linking

- **Static linking** – system libraries and program code combined by the loader into the binary program image
- Dynamic linking –linking postponed until execution time
- Small piece of code, **stub**, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
 - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**
- Consider applicability to patching system libraries
 - Versioning may be needed





Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk





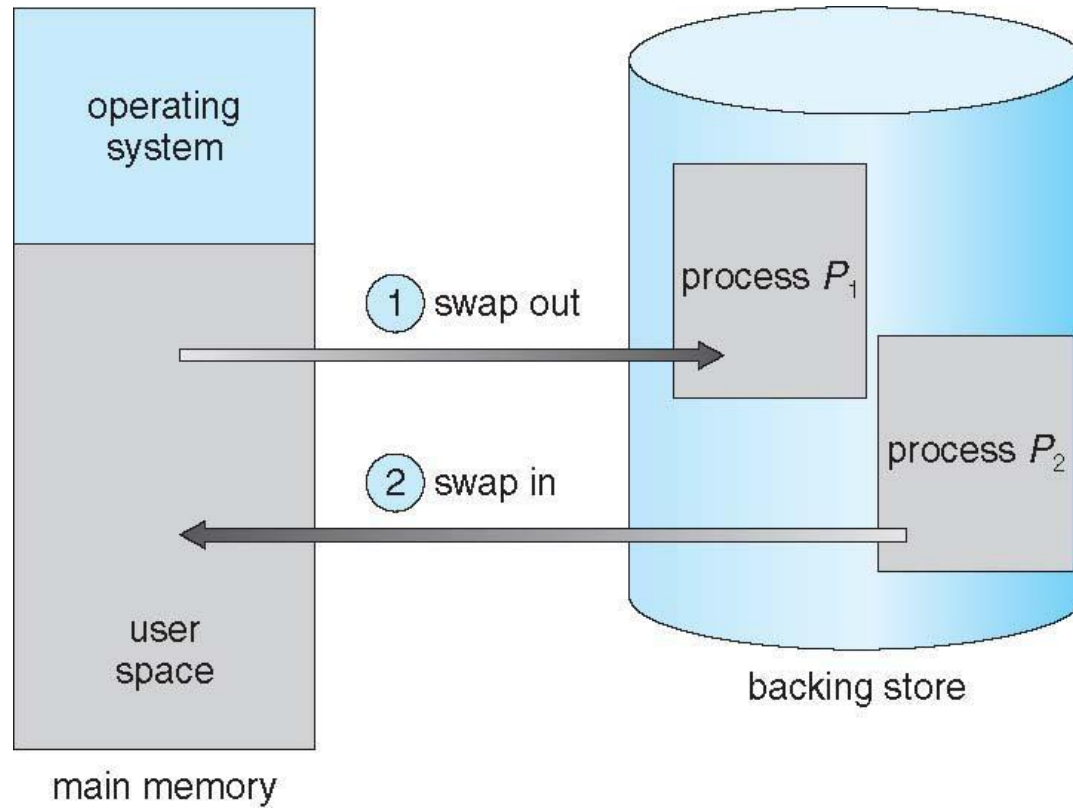
Swapping (Cont.)

- Does the swapped out process need to swap back in to same physical addresses?
- Depends on address binding method
 - Plus consider pending I/O to / from process memory space
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
 - Swapping normally disabled
 - Started if more than threshold amount of memory allocated
 - Disabled again once memory demand reduced below threshold





Schematic View of Swapping





Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
 - Swap out time of 2000 ms
 - Plus swap in of same sized process
 - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
 - System calls to inform OS of memory use via `request_memory()` and `release_memory()`





Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
 - Pending I/O – can't swap out as I/O would occur to wrong process
 - Or always transfer I/O to kernel space, then to I/O device
 - 4 Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
 - But modified version common
 - 4 Swap only when free memory extremely low





Swapping on Mobile Systems

- Not typically supported
 - Flash memory based
 - 4 Small amount of space
 - 4 Limited number of write cycles
 - 4 Poor throughput between flash memory and CPU on mobile platform
- Instead use other methods to free memory if low
 - iOS **asks** apps to voluntarily relinquish allocated memory
 - 4 Read-only data thrown out and reloaded from flash if needed
 - 4 Failure to free can result in termination
 - Android terminates apps if low free memory, but first writes **application state** to flash for fast restart
 - Both OSes support paging as discussed below





Contiguous Allocation

- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- Contiguous allocation is one early method
- Main memory usually into two **partitions**:
 - Resident operating system, usually held in low memory with interrupt vector
 - User processes then held in high memory
 - Each process contained in single contiguous section of memory





Contiguous Allocation (Cont.)

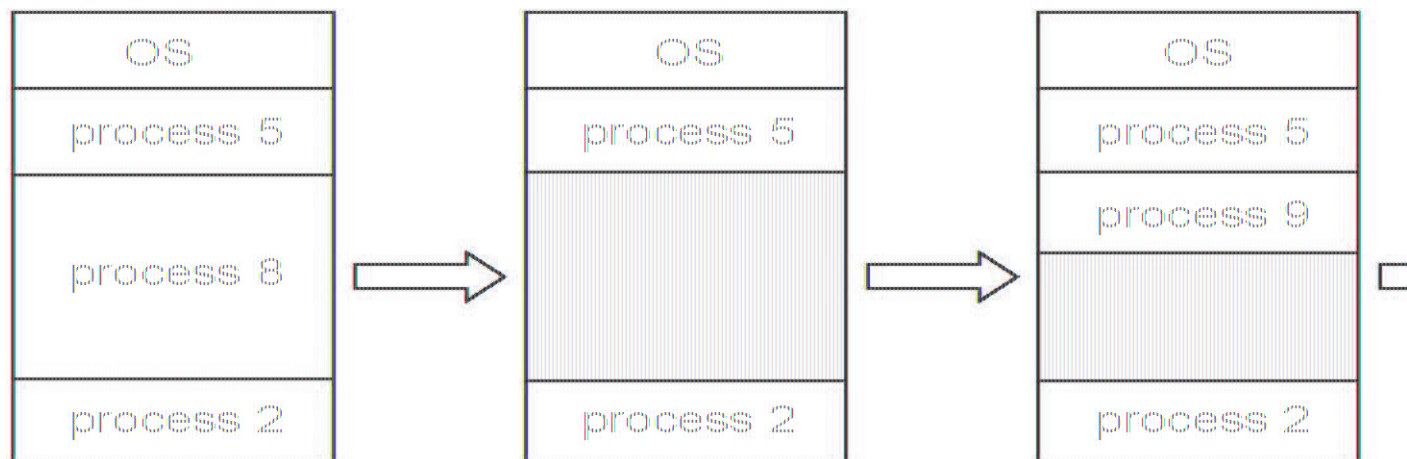
- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
 - Base register contains value of smallest physical address
 - Limit register contains range of logical addresses – each logical address must be less than the limit register
 - MMU maps logical address *dynamically*
 - Can then allow actions such as kernel code being **transient** and kernel changing size





Multiple-partition allocation

- Multiple-partition allocation
 - Degree of multiprogramming limited by number of partitions
 - **Variable-partition** sizes for efficiency (sized to a given process' needs)
 - **Hole** – block of available memory; holes of various size are scattered throughout memory
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it
 - Process exiting frees its partition, adjacent free partitions combined
 - Operating system maintains information about:
 - a) allocated partitions b) free partitions (hole)





Contiguous Memory Allocation

Fixed Size Partition: Fixed number of partitions

P1 - 5 MB, P2 - 4 MB, P3 - 15 MB, P4 - 13 MB

OS
4 MB
6 MB
20 MB
10 MB

OS
4 MB
6 MB
20 MB
10 MB

One block must contain only one process.

Problems:

- i. Internal Fragmentation
- ii. External Fragmentation





Contiguous Memory Allocation

Variable Size Partition: Partitions are created according to the requirements

P1 - 10 MB, P2 - 15 MB, P3 - 20 MB, P4 - 7 MB

OS
10 MB
15 MB
20 MB
5 MB

OS
7 MB
3 MB
6 MB
20 MB
10 MB

New Process
P5 - 8 MB

Problem: External Fragmentation

Possible Solution: Compaction by shifting processes but not convenient to apply





Fixed Size Partitioning

P1 - 20 MB, P2 - 12 MB, P3 - 15 MB, P4 - 10 MB, P5 - 14 MB

Process	First Fit
P2 -3	15 MB
P1 - 5	25 MB
P4 - 0	10 MB
	13 MB
P3 - 2	17 MB

Process	Next Fit
P4 - 5	15 MB
P1 - 5	25 MB
	10 MB
P2 - 1	13 MB
P3 - 2	17 MB

Process	Worst Fit
P3 - 0	15 MB
P1 - 5	25 MB
	10 MB
P4 - 3	13 MB
P2 - 5	17 MB

Scans every block and finds the best one.	
Reduces the amount of internal fragmentation.	
Process	Best Fit
P3 - 0	15 MB
P1 - 5	25 MB
P4 - 0	10 MB
P2 - 1	13 MB
P5 - 3	17 MB





Variable Size Partitioning

P1 - 20 MB, P2 - 12 MB, P3 - 15 MB, P4 - 10 MB, P5 - 14 MB

https://youtu.be/XmjG7E_x6pM?feature=shared

Process	First Fit
P2 - 3	15 MB
P1 - 5	25 MB
P4 - 0	10 MB
	13 MB
P3 - 2	17 MB

Process	Next Fit
P4 - 5	15 MB
P1 - 5	25 MB
	10 MB
P2 - 1	13 MB
P3 - 2	17 MB

Process	Worst Fit
P3 - 0	15 MB
P1 - 5	25 MB
	10 MB
P4 - 3	13 MB
P2 - 5	17 MB

Scans every block and finds the best one.	
Reduces the amount of internal fragmentation.	
Process	Best Fit
P3 - 0	15 MB
P1 - 5	25 MB
P4 - 0	10 MB
P2 - 1	13 MB
P5 - 3	17 MB





Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes?

- **First-fit**: Allocate the **first** hole that is big enough
- **Best-fit**: Allocate the **smallest** hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
- **Worst-fit**: Allocate the **largest** hole; must also search entire list
 - Produces the largest leftover hole

First-fit and best-fit better than worst-fit in terms of speed and storage utilization





Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation
 - $1/3$ may be unusable -> **50-percent rule**





Fragmentation (Cont.)

- Reduce external fragmentation by **compaction**
 - Shuffle memory contents to place all free memory together in one large block
 - Compaction is possible *only* if relocation is dynamic, and is done at execution time
 - I/O problem
 - 4 Latch job in memory while it is involved in I/O
 - 4 Do I/O only into OS buffers
- Now consider that backing store has same fragmentation problems





Paging

- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
 - Avoids external fragmentation
 - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
 - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size **N** pages, need to find **N** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- **Every process has its own page table**
- Backing store likewise split into pages
- Still have Internal fragmentation





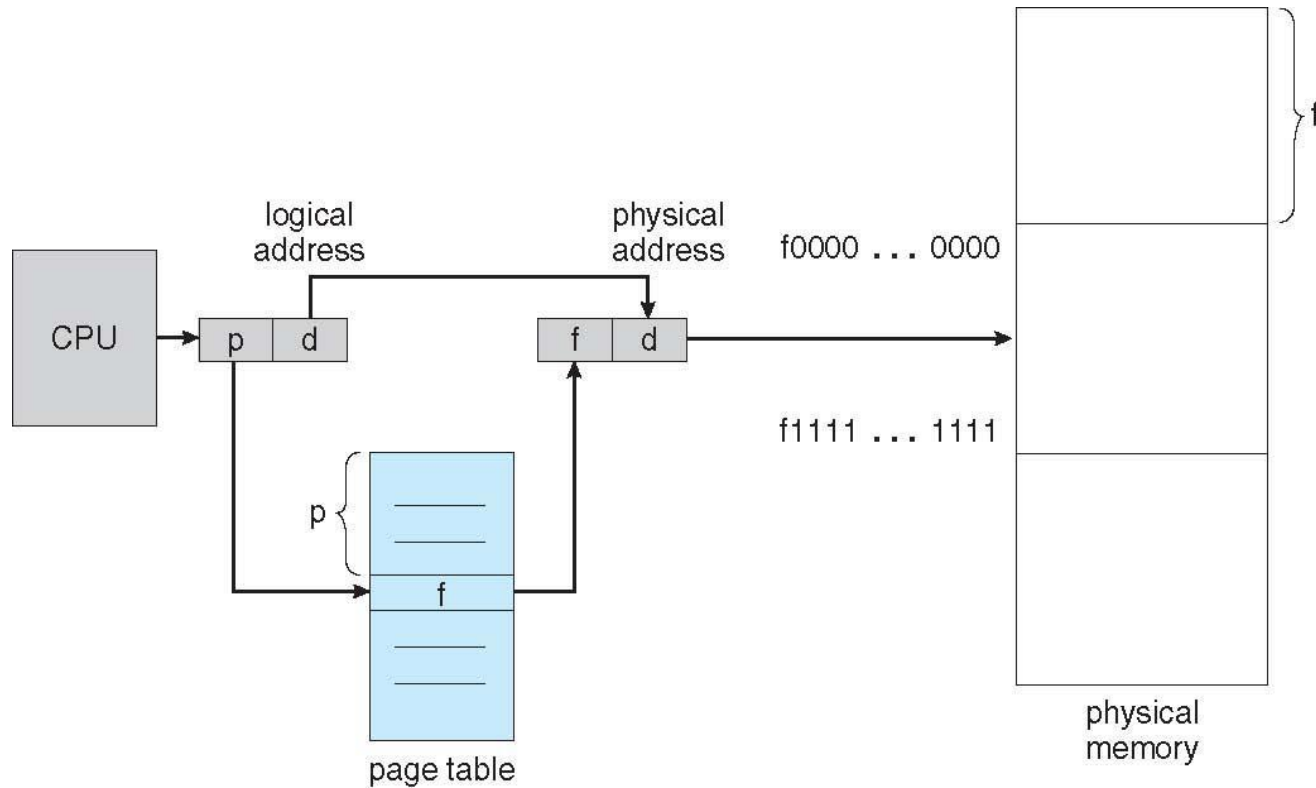
Address Translation Scheme

- Address generated by CPU (logical address) is divided into:
 - **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit
 - For given logical address space 2^m and Page Size 2^n
 - Main memory size / Frame size = number of frames
 - Page No (p) = Logical Address / Page size
 - Offset (d) = Logical Address % Page size
 - Physical Address = (Frame No * Page Size) + Offset
- Content of Page Offset and Frame Offset will be the same always, Offset stores the location in page or frame





Paging Hardware



p	page	o	f
p	d		
m	n		





Paging Hardware

Pg Size 2kb

Process 1		
Pg No	Instructions	Pg Offset
S0	Int 1	0
	Int 2	1
S1	Int 3	2
	Int 4	3
S2	Int 5	4
		5

Page Table

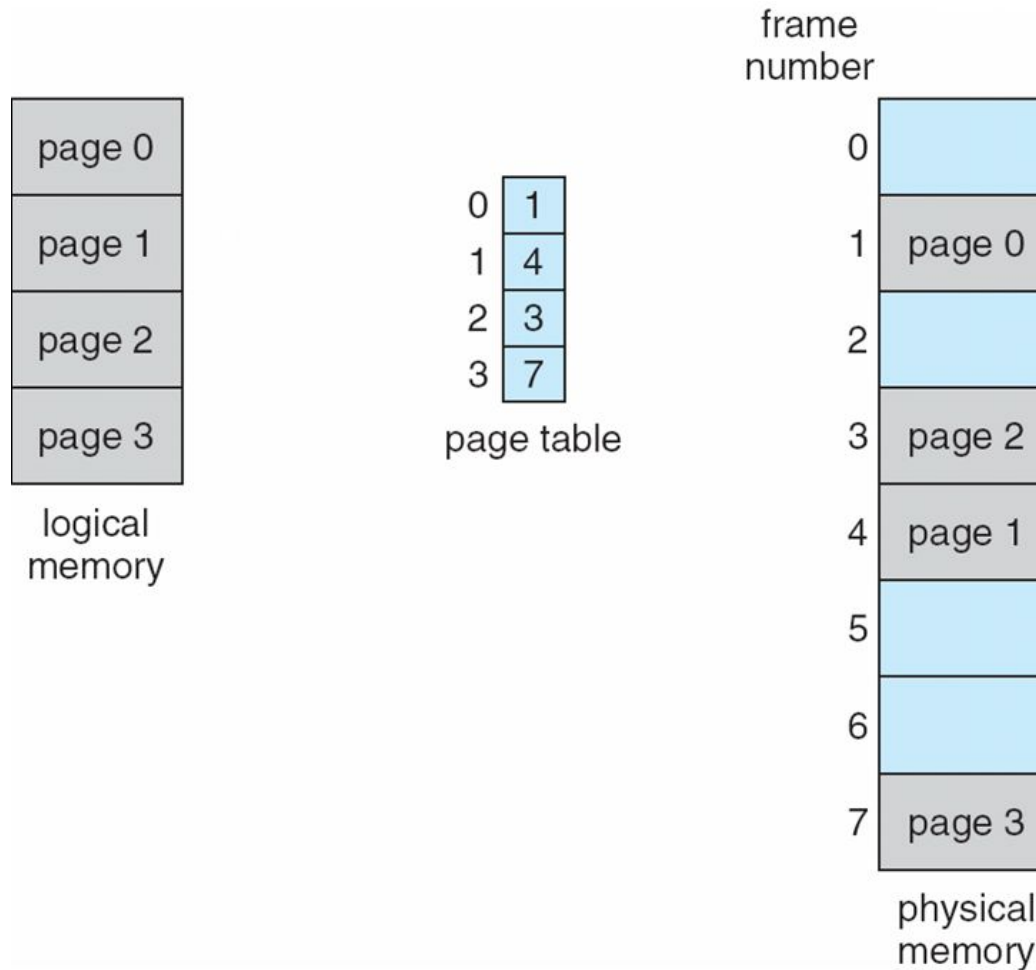
Fr Offset	Inst.	Fr No
0 1		F0
2 3	Int 3 Int 4	F1
4 5		F2
6 7		F3
8 9	Int 1 Int 2	F4
10 11		F5
12 13	Int 5	F6

Primary Memory



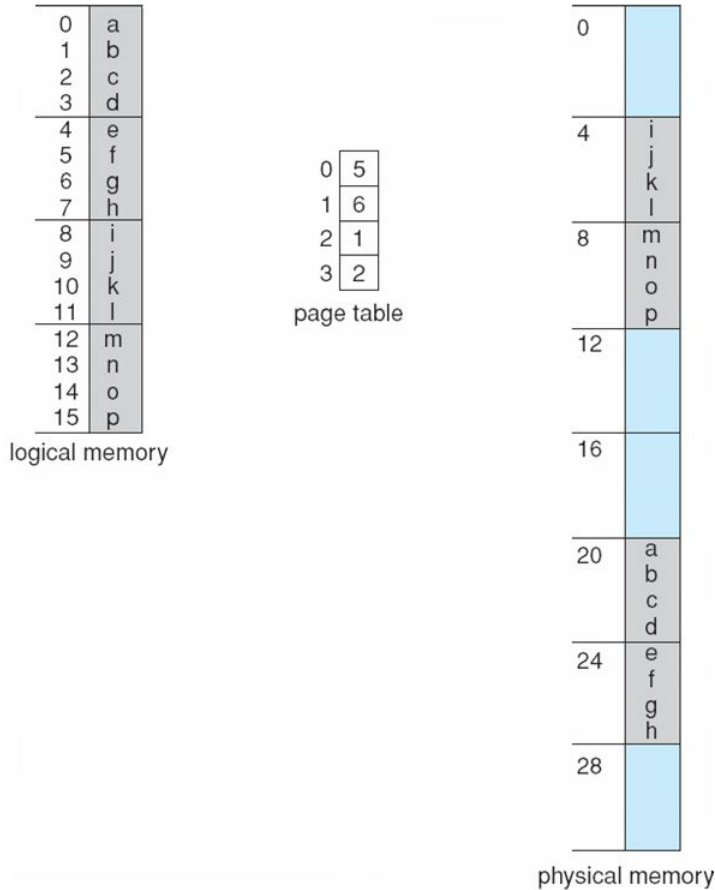


Paging Model of Logical and Physical Memory





Paging Example



$n=2$ and $m=4$ 32-byte memory and 4-byte pages





Paging (Cont.)

- Calculating internal fragmentation
 - Page size = 2,048 bytes
 - Process size = 72,766 bytes
 - 35 pages + 1,086 bytes
 - Internal fragmentation of $2,048 - 1,086 = 962$ bytes
 - Worst case fragmentation = 1 frame – 1 byte
 - On average fragmentation = $1 / 2$ frame size
 - So small frame sizes desirable?
 - But each page table entry takes memory to track
 - Page sizes growing over time
 - 4 Solaris supports two page sizes – 8 KB and 4 MB
- Process view and physical memory now very different
- By implementation process can only access its own memory





Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**





Implementation of Page Table (Cont.)

- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process to provide address-space protection for that process
 - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
 - Replacement policies must be considered
 - Some entries can be **wired down** for permanent fast access





Associative Memory

- Associative memory – parallel search

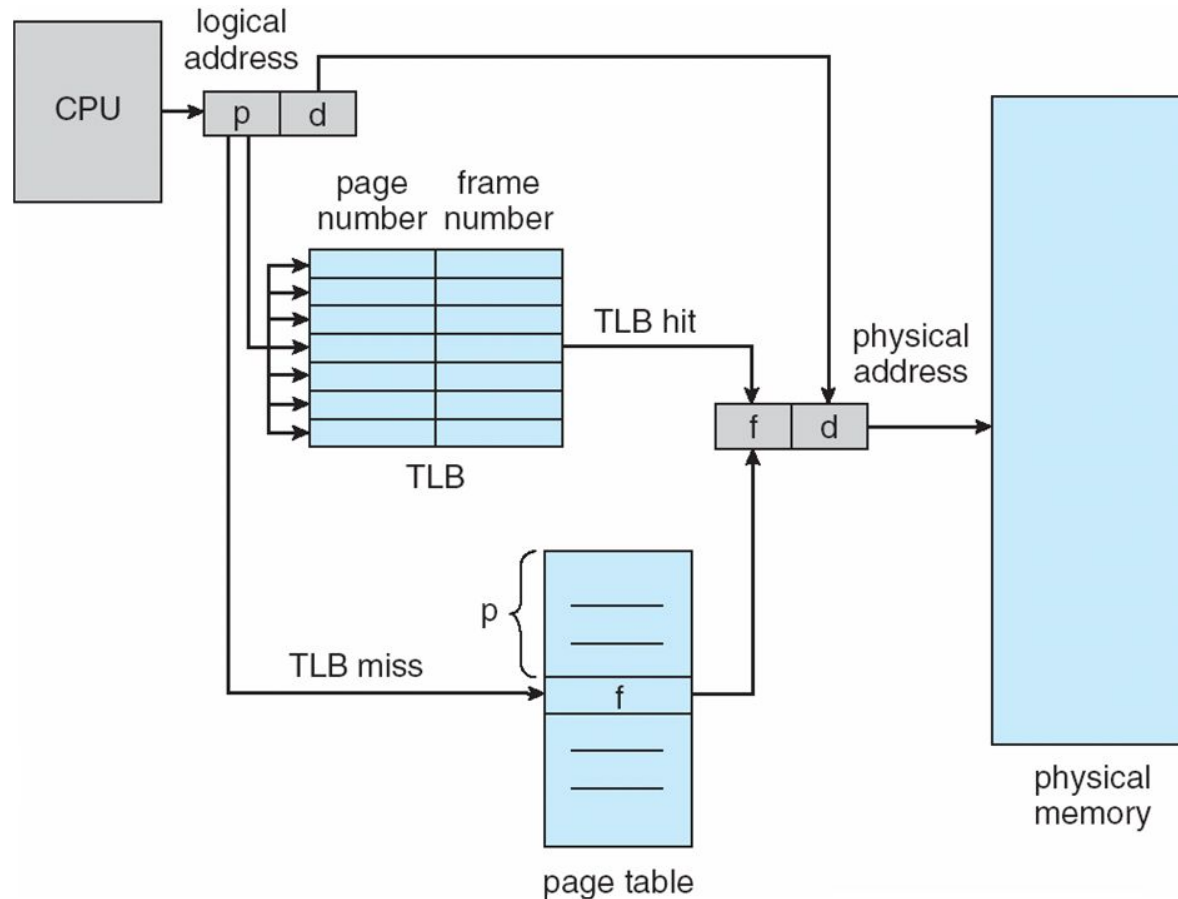
Page #	Frame #

- Address translation (p, d)
 - If p is in associative register, get frame # out
 - Otherwise get frame # from page table in memory





Paging Hardware With TLB





Paging Hardware With TLB

TLB access time is 25ns and primary memory access time is 150ns. If hit ratio is 70% then find out effective access time (EAT).

$$\begin{aligned} \text{EAT} &= \text{hit}(\text{TLB} + \text{PM}) + \text{miss}(\text{TLB} + \text{PM} + \text{PM}) \\ &= \text{hit}(\text{TLB} + \text{PM}) + (1 - \text{hit})(\text{TLB} + 2\text{PM}) \\ &= 0.7(25 + 150) + (1 - 0.7)(25 + 2 * 150) \\ &= 220\text{ns} \end{aligned}$$

Without TLB,

$$\begin{aligned} \text{EAT} &= 2\text{PM} \\ &= 2 * 150 \\ &= 300\text{ns} \end{aligned}$$

Note:

If TLB hit ratio is increased then EAT will be decreased and vice versa.

Using hit ratio 80%, EAT = 205ns!





TLB: Page Fault

TLB access time is 40ns and primary memory access is 137ns where page fault is 45%. If the hit ratio is 65% and page fault service time is 1000ns then find out effective access time.

$$\begin{aligned} \text{EAT} &= \text{hit}(\text{TLB} + \text{PM}) + \text{miss}(\text{TLB} + \text{PM} + \text{Page Fault} * \text{Service Time} + \text{No} \\ &\quad \text{Page Fault} * \text{PM}) \\ &= 0.65(40 + 137) + (1 - 0.65)(40 + 137 + 0.45 * 1000 + 0.55 * 137) \\ &= 360.85\text{ns} \end{aligned}$$





Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
 - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
 - Or use **page-table length register (PTLR)**
- Any violations result in a trap to the kernel





Valid (v) or Invalid (i) Bit In A Page Table

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page <i>n</i>





Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments

A segment is a logical unit such as,

main program

procedure

function

method

object

local variables, global variables

common block

stack

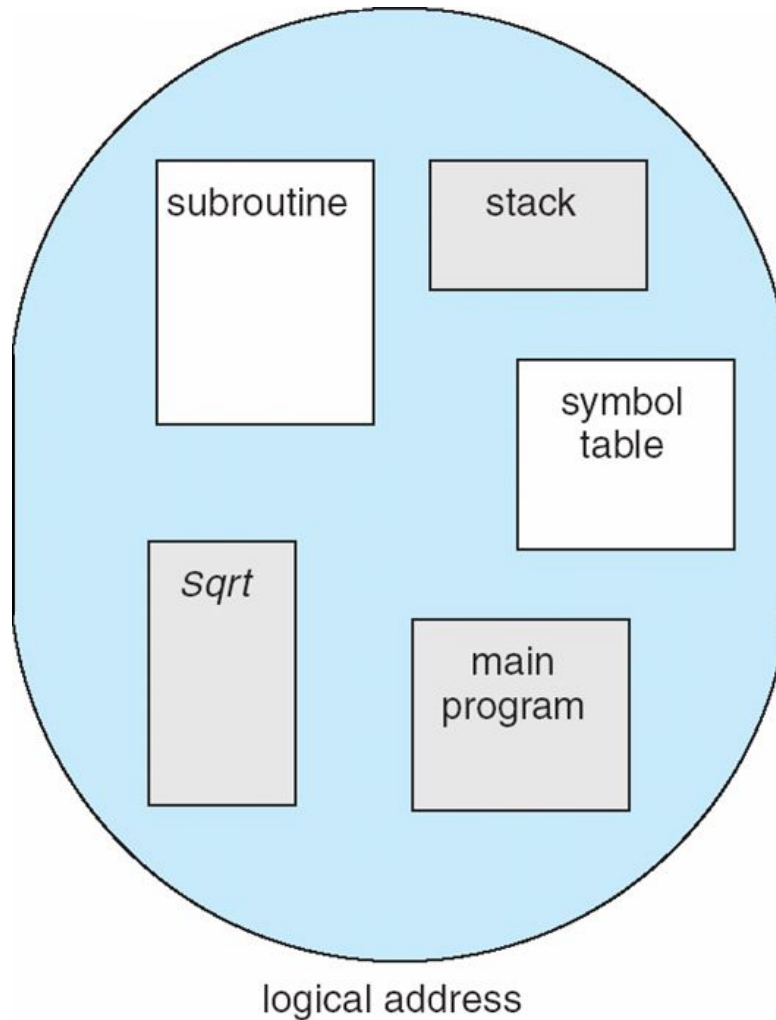
symbol table

arrays



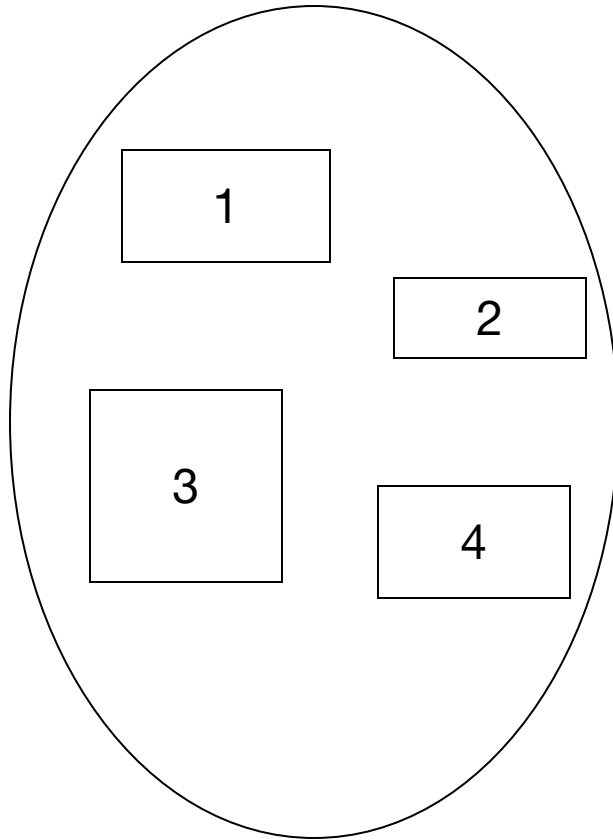


User's View of a Program

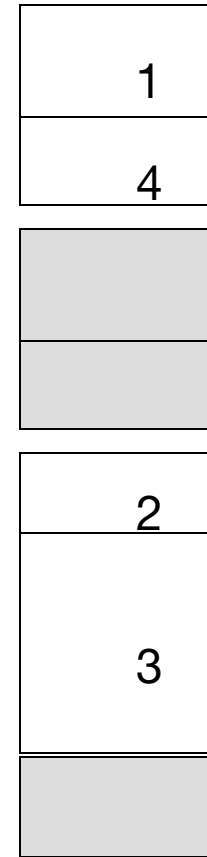




Logical View of Segmentation



user space



physical memory space





Segmentation Hardware

Sg No	Limit	Base
S0	1200 - 1100 = 100	1100
S1	40	1260
S2	80	1000
S3	10	1200

Segment Table

Primary Memory

Base Address	OS
Here the values represent the starting address of these blocks	
1000	S2
1080	////////////////
1100	S0
1200	S3
1210	////////////////
1260	S1
1300	Sg Table

STBR+Base Add -> Finds the Segment in Primary Memory from Sg Table such as, CPU wants to access s = 0 address so STBR -> 1300 + S -> 0 = 1300 location of S0 in Primary Memory





Segmentation: Logical Address to Physical Address

Sg No	Base Address	Limit
0	1500	300
1	500	250
2	1400	70
3	1200	150
4	150	50

Segment Table

Lg Add	$d < \text{limit}$	base+offset	Phy Add
0, 280	$280 < 300$	$1500 + 280$	1780
1, 252	$252 > 250$	-	-
1, 180	$150 < 250$	$500 + 180$	680
2, 65	$65 < 70$	$1400 + 65$	1465
3, 160	$160 < 150$	-	-
4, 40	$40 < 50$	$150 + 40$	190

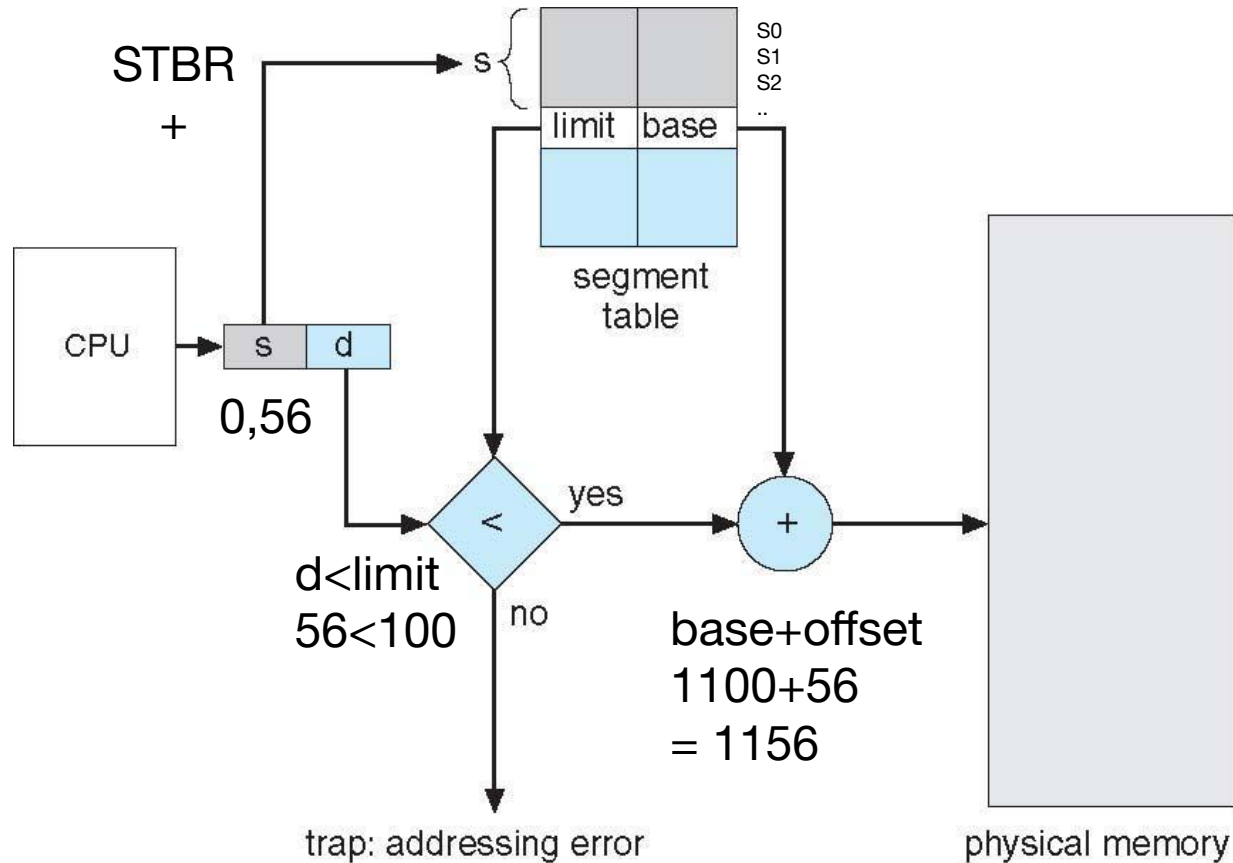
Disadvantages of Segmentation:

- Variable size partition so external fragmentation will exist
- Internal fragmentation may exist also





Segmentation Hardware Architecture



STBR -> Segment Table Base Register
Base -> Primary Memory Address
Limit -> Segment Size





Segmentation Architecture

- Logical address consists of a two tuple:
 <segment-number, offset>,
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - **base** – contains the starting physical address where the segments reside in memory
 - **limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;
 segment number **s** is legal if **s** < **STLR**





Segmentation Architecture (Cont.)

- Protection
 - With each entry in segment table associate:
 - 4 validation bit = 0 $\Rightarrow$ illegal segment
 - 4 read/write/execute privileges
- Protection bits associated with segments; code sharing occurs at segment level
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem
- A segmentation example is shown in the following diagram





Paging vs Segmentation

- Paging -> Fixed size partition in main memory
- Segmentation -> Variable size partition in main memory

Segment Table contains Base address (Starting address) and Limit (Size)

Video Resource: <https://youtu.be/AjunUkCa5cU?feature=shared>



End of Chapter 8

